Alzikrayat – Architecture Report

Name: Lina Mohammed Widaa

Course: Advanced Web Technologies

Project 1: Photo Sharing Application (Alzikrayat)

1. Introduction

Alzikrayat is a small photo sharing website where a user can make an account, upload photos, look at other people's photos in a gallery and write comments on them. I built it with PHP and MySQL.

The main goal of this project was not really the photos, it was to build the MVC structure and the router myself without using a framework like Laravel. So everything (the routing, the SQL, the sessions) is written by hand. This report explains how I organised the code and the decisions I made.

2. Three-Tier Architecture

I divided the project into three layers so that each part has its own job:

- **Presentation tier** – the views (the HTML pages the user sees). These are in the `views/` folder and use Bootstrap for the design.
- **Application tier** – the server logic: the router and the controllers. This layer takes the request, checks the input and decides what to show.
- **Data tier** – the models and the MySQL database. Only the models talk to the database.

The reason for splitting it like this is to keep things clean. For example a view never runs a SQL query directly, it only prints the data the controller gives it, and a model never prints HTML.

3. MVC Pattern

Inside the application the code follows Model–View–Controller:

- **Model:** one class for each table (`User`, `Photo`, `Comment`). They all extend a base `Model` class that opens the PDO connection, so each model only has to write its own queries.
- **View:** simple PHP templates that just display data. I made a shared layout (`header.php` and `footer.php`) that holds the navbar and footer so every page looks the same and I don't repeat the code.
- **Controller:** one class per feature (`AuthController`, `PhotoController`, `CommentController`). A controller reads the request, calls the model and then renders a view or redirects. They extend a base `Controller` that has two helpers, `view()` and `redirect()`.

4. The Router

This is the part I spent the most time on. Instead of a framework doing the routing, I wrote a `Router` class. In `public/index.php` I register the routes like this:

```
$router->get('/photo/{id}', function ($id) {
    (new PhotoController())->show($id);
});
```

When a request comes in, the router loops over the routes and turns the `{id}` part into a regular expression so it can match a real URL like `/photo/5`:

```
$pattern = preg_replace('/\{([^}]+)\}/', '([^/]+)', $route);
$pattern = '#^' . $pattern . '$#';
```

If the pattern matches, the captured value (the id) is passed to the function. If nothing matches it returns a 404.

One problem I had here: my project runs in a folder (`http://localhost/alzikrayat/public/...`), so the URL path always started with `/alzikrayat/public` and none of my routes matched at the beginning. I fixed it by removing that base part from the path before matching, and I detect the base folder automatically so it also works with the PHP built-in server.

5. Database

There are three tables: `users`, `photos` and `comments`. Every table has an auto-increment integer `id` as the primary key. `photos.user_id` is a foreign key to `users`, and `comments` has foreign keys to both `photos` and `users`.

I used `ON DELETE CASCADE` on the foreign keys. This means if a user is deleted their photos and comments go with them, and if a photo is deleted its comments are removed too, so I don't end up with rows that point to nothing. I also put indexes on the foreign key columns. The password column only stores the hashed password, never the real one.

6. Authentication and Sessions

When a user registers, I hash the password with `password_hash($password, PASSWORD_BCRYPT)` before saving it, so the real password is never in the database. At login I look the user up by email and check the password with `password_verify()`. If it's correct I save the user's id in `$_SESSION`, which keeps them logged in between pages.

I also set a cookie called `last_login` with the current time and a 7-day life. The next time the login page opens I read this cookie and show "Last login from this computer was ...". The navbar checks the session on every page: if the user is logged in it shows "Hi <name>" and a Logout button, otherwise it shows "Please Login".

7. Validation and Security

Every form is validated on three levels:

1. **HTML5** – `required`, `type="email"`, `maxlength` and so on.
2. **JavaScript** – a small script checks the values before the form is sent.
3. **Server-side PHP** – the controller checks everything again, because the first two can be turned off by the user.

For security I used PDO prepared statements with placeholders everywhere, so the user input is never put directly into the SQL (this stops SQL injection). Before printing any data in a view I escape it with `htmlspecialchars()` to stop XSS. For deleting a photo I check the owner twice: the controller compares the photo's `user_id` with the logged-in user, and the delete query also has `AND user_id = ?`, so a user can only delete their own photos.

8. Problems I Faced

The most annoying bug was that some of my PHP files had an empty line before the `<?php` tag. That empty line was being sent to the browser, and after that PHP would not let me set the cookie or redirect ("headers already sent"). It took me a while to find because the code looked fine. After I removed the blank lines the last-login cookie and the redirects worked.

The router base path (section 4) was the other thing that confused me at first.

9. Conclusion

In the end the project does what was asked: a photo sharing app with my own MVC framework and a hand-written router, raw SQL with prepared statements, sessions and cookies for login, three layers of validation and a responsive Bootstrap design. The most useful thing for me was seeing how routing, sessions and cookies actually work instead of a framework hiding them.